Java OOP Laboratory Exercise

Campus Café Ordering System

Written Report

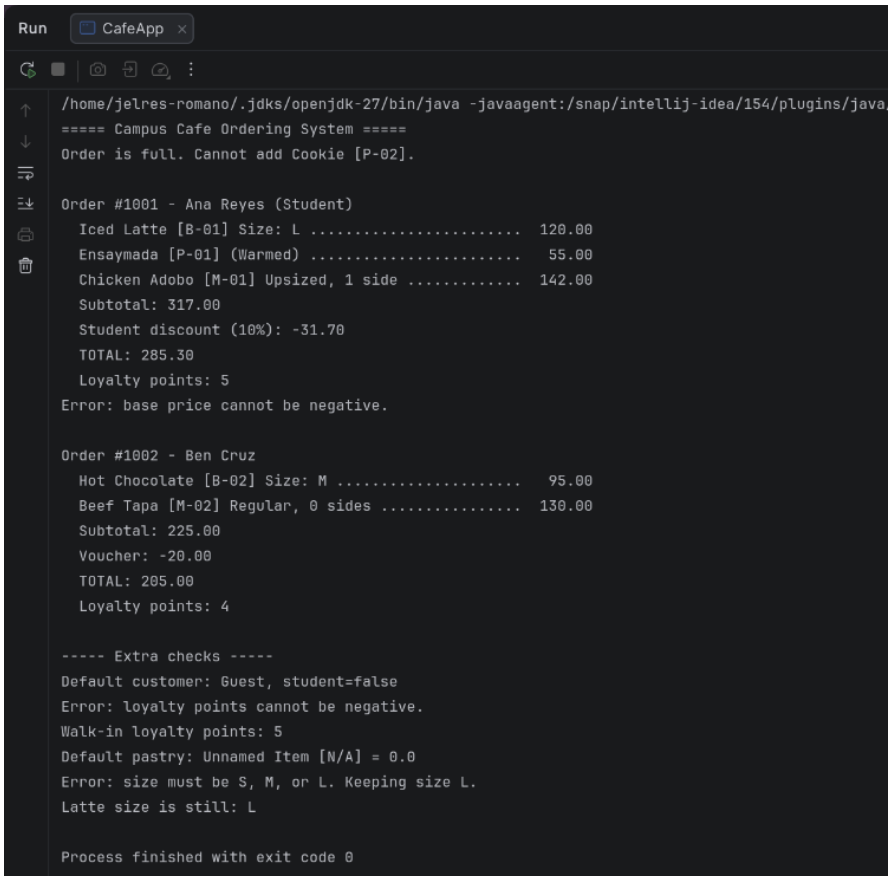
Names:	Jelres C. Romano Ildefonso, Providencia Jr.
Section:	ITS 202 OOP - Section C
Date:	November 5, 2026
Exercise:	Java OOP Laboratory Exercise: Campus Café Ordering System

1. Concept Explanation

Concept	Where and why we used it
if Statement	Pastry.computePrice() adds the warming fee only when warmed is true; Meal.computePrice() adds the upsize fee only when upsized is true. Both are optional extras, so a single if without an else is enough.
if-else Statement	Used where there are two or more outcomes: MenuItem.setBasePrice() (accept or reject), Beverage.computePrice() (if-else-if chain on size), Order.addItem() (full or place item) and Order.computeTotal() (never below zero).
Classes and Objects	Seven classes, each in its own file: MenuItem, Beverage, Pastry, Meal, Customer, Order and CafeApp. CafeApp.main() creates the objects (two customers, several menu items, two orders).
Attributes and Behaviors	Every class keeps its data in instance variables (e.g., size, warmed, loyaltyPoints) and works on it through instance methods (e.g., computePrice(), addPoints(), printReceipt()).
Object Interaction	Order stores a Customer and up to three MenuItem objects and calls their methods. printReceipt() calls customer.addPoints(), so the order updates the customer's loyalty points.
Naming Conventions	PascalCase for classes (MenuItem), camelCase for methods and variables (computeSubtotal, itemCount) and UPPER_SNAKE_CASE for constants (STUDENT_DISCOUNT_RATE, WARMING_FEE).
Default Constructor	MenuItem(), Beverage(), Pastry(), Meal() and Customer() set safe starting values (e.g., "Unnamed Item", size "S", name "Guest") so an object is always valid.
Parameterized Constructor	The MenuItem subclasses, Customer and Order accept their data on creation. Subclasses call super(...) to pass shared data to MenuItem, and values go through the validating setters.
Constructor Overloading	Customer has three constructors: Customer(), Customer(String name) and Customer(String name, boolean student). The longer ones reuse the shorter ones with this(...) to avoid repeated code.
public / private / protected	All fields are private except name and basePrice in MenuItem, which are protected so subclasses can use them in computePrice(). Behavior is exposed through public methods; there are no public fields.
Getters, Setters, Data Hiding	Outside code can only change data through setters that check it first: negative prices, negative extra sides and negative loyalty points are rejected, and setSize() accepts only S, M or L.
Method Overloading	Customer.addPoints(int) adds points directly while addPoints(double) converts pesos spent into points (1 per 50). Order.computeTotal() and computeTotal(double voucher) differ by the voucher parameter.
Method Overriding	Beverage, Pastry and Meal override computePrice() and getDescription() with @Override, so each type prices and describes itself differently while Order treats them all as MenuItem (polymorphism).
Abstraction	MenuItem is abstract with the abstract method computePrice(). It cannot be instantiated, and every subclass is forced to supply its own pricing rule.

2. Program Output

Screenshot of the program running in IntelliJ IDEA (`CafeApp`, exit code 0); it also compiles with `javac *.java` and runs with `java CafeApp`. The values match the sample: Order #1001 totals 285.30 (5 points) and Order #1002 totals 205.00 (4 points). The output also shows the refused fourth item and the rejected negative price.



3. Challenges Encountered

No collections allowed. An order has to hold several items without an ArrayList. We used three separate fields (item1, item2, item3) with an itemCount, and made addItem() fill the first empty slot, which also made the three-item limit easy to check.

Keeping constructors from accepting bad data. A parameterized constructor could bypass the validation in the setters. We first set safe defaults (this() or super()) and then passed the values through the setters, so an invalid price or size is rejected even at creation.

Matching the expected totals. Loyalty points had to come out as 5 and 4, so we computed them from the amount actually paid (after the discount and voucher), using integer conversion to drop the leftover pesos. We also kept the output plain ASCII so it prints correctly on any console.